VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

B.Tech – Computer Science & Engineering | A9503 – Data Structures | VCE-R25

Unit – I Handout

Introduction to C++ and Data Structures

C++ Basics	OOP Concepts	Functions	Templates	STL

Mr. Ganesh Deshmukh — Assistant Professor, CSE

CO Mapped: A9503.1 | CIE: 40 | SEE: 60 | Total: 100

Table of Contents

1. C++ Program Structure
2. Classes and Objects
3. Dynamic Memory Allocation
4. Constructors and Destructors
5. Access Specifiers, Encapsulation & Abstraction
6. Pointers and References
7. Functions – Inline & Overloading
8. Templates
9. Standard Template Library (STL)
10. Iterators and Algorithms
11. Exam Preparation

1. C++ Program Structure

A C++ program is organized into **translation units**. Each file passes through the **preprocessor** → **compiler** → **linker** pipeline before producing an executable.

```
// hello.cpp

#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

Key Elements of Every C++ Program

#include — include header files (standard or user-defined)
using namespace std — avoids prefixing std:: on every standard name
int main() — entry point; must return int (0 = success)
Statements end with semicolons; blocks enclosed in { }
// single-line comment | /* multi-line comment */

Compilation Pipeline

Stage	Tool	Input	Output
Preprocessor	Cpp	source.cpp	source.i (expanded)
Compiler	g++	source.i	source.s (assembly)
Assembler	As	source.s	source.o (object)
Linker	Ld	*.o + libs	executable

2. Classes and Objects

A **class** is a user-defined type that bundles **data (attributes)** and **functions (methods)**. An **object** is an instance of a class.

```
// Student.cpp

class Student {
private:
    int rollNo;
    string name;
public:
    void setData(int r, string n) {
        rollNo = r;  name = n;
    }
    void display() {
        cout << rollNo << "  " << name;
    }
};

int main() {
    Student s1;          // object created on stack
    s1.setData(101, "Alice");
    s1.display();
    return 0;
}
```

Class vs Object — Quick Reference

Class = Blueprint / Template (defined once, occupies no memory by itself)
Object = Instance of a class (occupies memory, one per instance created)
Member variables — data stored in each object
Member functions — operations that can be performed on the object
Dot operator (.) — access members of an object: s1.display()
Arrow operator (->) — access members via pointer: ptr->display()

Structure (C)	Class (C++)
Members are public by default	Members are private by default
No member functions allowed	Member functions fully supported
No access control	public / private / protected
No constructors/destructors	Full constructor/destructor support

3. Dynamic Memory Allocation (new & delete)

C++ uses **new** to allocate memory on the **heap** at runtime and **delete** to free it. Forgetting **delete** causes a **memory leak**.

```
// dynamic_memory.cpp

// Allocate a single int
int* p = new int(42);
delete p;           // free single variable

// Allocate an array of 5 ints
int* arr = new int[5];
arr[0] = 10;  arr[1] = 20;
delete[] arr;      // free array — ALWAYS use delete[]

// Allocate an object
Student* sp = new Student();
sp->setData(101, "Bob");
delete sp;
```

Stack vs Heap Memory

Stack — automatic, fast, limited size, LIFO; variables destroyed when scope ends
Heap — manual (new/delete), large, persistent; programmer controls lifetime
new returns a pointer; must store it or memory is unreachable (leak!)
Always pair: new → delete and new[] → delete[]
Accessing freed memory = undefined behaviour (dangling pointer bug)

Operation	Syntax	Notes
Allocate single	int* p = new int(val)	Use delete p to free
Allocate array	int* a = new int[n]	Use delete[] a to free
Allocate object	Student* s = new Student()	Use delete s to free
Check failure	new throws std::bad_alloc	Wrap in try-catch if needed
Null pointer	int* p = nullptr	Always initialise pointers

4. Constructors and Destructors

Constructors initialise objects automatically when created. **Destructors** clean up when an object goes out of scope or is deleted.

```
// Box.cpp

class Box {
private:  double l, b, h;
public:
    // Default constructor
    Box() : l(1), b(1), h(1) { cout << "Box created\n"; }

    // Parameterised constructor
    Box(double x, double y, double z) : l(x), b(y), h(z) {}

    // Copy constructor
    Box(const Box& other) : l(other.l), b(other.b), h(other.h) {}

    // Destructor
    ~Box() { cout << "Box destroyed\n"; }

    double volume() { return l * b * h; }
};

int main() {
    Box b1;                  // default
    Box b2(3.0, 4.0, 5.0);  // parameterised
    Box b3(b2);              // copy
}  // destructors called automatically here
```

Type	Signature	Called When
Default	ClassName()	Object created with no arguments
Parameterised	ClassName(args)	Object created with arguments
Copy	ClassName(const T& obj)	Object initialised from another object
Destructor	~ClassName()	Object goes out of scope or delete called

Constructor Rules

Same name as class; no return type (not even void)
Can be overloaded (multiple constructors with different parameters)
Compiler provides a default constructor only if you define NONE
Member initialiser list ClassName(args) : member(value) is preferred over assignment in body
Destructors: only one per class; no parameters; no return type
Destructors are called in REVERSE order of construction

5. Access Specifiers, Encapsulation & Abstraction

5.1 Access Specifiers

Specifier	Accessible in same class?	Accessible in derived class?	Accessible outside class?
private	Yes	No	No
protected	Yes	Yes	No
public	Yes	Yes	Yes

5.2 Encapsulation

Encapsulation = bundling data + methods + restricting direct access.
Use **private** for data, **public** getter/setter functions to control access.

```
// BankAccount.cpp

class BankAccount {
private:  double balance;   // hidden from outside
public:
    BankAccount(double b) : balance(b) {}
    void deposit(double amt) { if(amt>0) balance += amt; }
    void withdraw(double amt) { if(amt<=balance) balance -= amt; }
    double getBalance() { return balance; }
};
```

5.3 Abstraction

Abstraction = hiding implementation details, exposing only the interface.
Achieved via **abstract classes** (with pure virtual functions) or simply by providing **public interfaces** and keeping internals private.

Encapsulation vs Abstraction

Encapsulation — HOW data is protected (private members + public methods)
Abstraction — WHAT is shown to the user (interface hiding complexity)
Both together achieve data hiding and clean, modular design
Example: cout hides all I/O buffering (abstraction); cin's buffer is private (encapsulation)

6. Pointers and References

6.1 Pointers

A **pointer** stores the **memory address** of another variable. Operators: & (address-of) and * (dereference).

```
// pointers.cpp

int x = 10;
int* ptr = &x;        // ptr holds address of x
cout << *ptr;      // outputs 10 (dereference)
*ptr = 99;         // x is now 99

// Pointer arithmetic
int arr[] = {1,2,3};
int* p = arr;
cout << *(p+1);    // outputs 2

// Pointer to object
Box* bp = new Box(2,3,4);
bp->volume();      // arrow operator
delete bp;
```

6.2 References

A **reference** is an alias for an existing variable. Must be initialised at declaration and cannot be rebound.

```
// references.cpp

int a = 5;
int& ref = a;     // ref is alias for a
ref = 20;          // a is now 20

// Reference as function parameter (pass by reference)
void swap(int& x, int& y) {
    int t = x;  x = y;  y = t;
}
int main() { int p=3, q=7; swap(p,q); }
```

Pointer	Reference
Can be null (nullptr)	Cannot be null
Can be rebound to another variable	Bound at init; cannot rebind
Needs * and & operators	Transparent — used like normal variable

Pointer	Reference
Supports arithmetic (ptr++)	No arithmetic
Used for arrays, dynamic memory	Used for function params, aliases

7. Functions — Inline & Overloading

7.1 Regular Functions

Functions promote **code reuse**, **modularity**, and **readability**.
Syntax: **return_type function_name (parameter_list) { body }**

```
// functions.cpp

// Function declaration (prototype)
int add(int a, int b);

// Function definition
int add(int a, int b) { return a + b; }

// Default parameter values
double power(double base, int exp = 2) {
    double result = 1;
    for(int i=0; i<exp; i++) result *= base;
    return result;
}

int main() {
    cout << add(3, 4);          // 7
    cout << power(3.0);         // 9.0 (exp defaults to 2)
    cout << power(2.0, 3);      // 8.0
}
```

7.2 Inline Functions

The **inline** keyword requests the compiler to replace each call with the function's body, eliminating function call overhead. Best for small, frequently-called functions.

```
// inline_func.cpp

inline int square(int x) { return x * x; }

// Compiler may expand: cout << square(5) → cout << 5*5
// No function call overhead for tiny computations
```

Inline Function Rules

Use inline for small, simple functions (1-5 lines)
inline is a hint only — compiler may ignore it
Inline functions are typically defined in header files
Complex functions with loops/recursion will NOT be inlined
Member functions defined inside a class body are implicitly inline

7.3 Function Overloading

C++ allows multiple functions with the **same name** but **different parameter lists**. The compiler selects the correct version based on argument types — this is **compile-time polymorphism**.

```
// overloading.cpp

int    area(int side)                        // square
    { return side * side; }

double area(double r)                       // circle
    { return 3.14159 * r * r; }

int    area(int l, int b)                   // rectangle
    { return l * b; }

int main() {
    cout << area(5);          // calls int version    → 25
    cout << area(3.14);       // calls double version
    cout << area(4, 6);       // calls two-int version → 24
}
```

Overloading Rules

Functions MUST differ in: number of parameters OR types of parameters
Return type alone is NOT sufficient to overload (compile error!)
Default arguments + overloading can cause ambiguity — be careful
Operator overloading is also possible (e.g., +, ==, <<)

8. Templates

Templates enable **generic programming** — write one function or class that works for any data type.
The compiler generates type-specific code at compile time (**template instantiation**).

8.1 Function Templates

```
// function_template.cpp

template <typename T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}

// Template with two types
template <typename T1, typename T2>
void display(T1 a, T2 b) {
    cout << a << " " << b;
}

int main() {
    cout << myMax(3, 7);            // T = int    → 7
    cout << myMax(3.5, 2.1);        // T = double → 3.5
    cout << myMax("apple", "mango");   // T = string
    display(42, "hello");           // T1=int, T2=string
}
```

8.2 Class Templates

```
// class_template.cpp

template <typename T>
class Stack {
private:
    T arr[100];
    int top;
public:
    Stack() : top(-1) {}
    void push(T val) { arr[++top] = val; }
    T pop()          { return arr[top--]; }
    bool isEmpty()   { return top == -1; }
};

int main() {
    Stack<int>    si;  si.push(10); si.push(20);
    Stack<double> sd;  sd.push(3.14);
    Stack<string>  ss;  ss.push("hi");
```

```
}
```

8.3 Template Specialisation

Full specialisation overrides template behaviour for a **specific type**.

```
// specialisation.cpp

template <typename T>
void print(T val) { cout << val; }  // generic

template <>                             // full specialisation for bool
void print<bool>(bool val) {
    cout << (val ? "true" : "false");
}
```

Template Summary

template <typename T> — declares a type parameter T
typename and class are interchangeable inside <> (use typename by convention)
Compiler generates separate code for each type used (instantiation)
Templates must be defined in header files (linker needs them at compile time)
Function templates: compiler deduces T from arguments automatically
Class templates: T must be specified explicitly Stack<int> Stack<double>

9. Standard Template Library (STL)

The STL provides **ready-made, generic, efficient** implementations of common data structures and algorithms. It has three pillars: **Containers**, **Iterators**, and **Algorithms**.

STL Architecture

Containers — store and manage collections of data (vector, list, map, set, ...)
Iterators — pointer-like objects to traverse containers uniformly
Algorithms — generic functions working via iterators (sort, find, count, ...)
Include: #include <vector> #include <list> #include <map> #include <algorithm>

9.1 vector (dynamic array)

Random access O(1) · push_back O(1) amortised · insert/erase O(n) · sequential memory

```
// vector_demo.cpp

#include <vector>
using namespace std;

vector<int> v = {10, 20, 30};
v.push_back(40);        // {10,20,30,40}
v.pop_back();           // {10,20,30}
v.insert(v.begin()+1, 99); // {10,99,20,30}
v.erase(v.begin());     // {99,20,30}
cout << v.size();       // 3
cout << v[1];           // 20  (direct index)
cout << v.at(1);        // 20  (bounds-checked)

// Range-based for loop
for(int x : v) cout << x << " ";
```

9.2 list (doubly linked list)

No random access · O(1) insert/delete anywhere · bidirectional traversal

```
// list_demo.cpp

#include <list>
list<int> lst = {1, 2, 3};
lst.push_front(0);      // {0,1,2,3}
lst.push_back(4);       // {0,1,2,3,4}
lst.pop_front();        // {1,2,3,4}
lst.reverse();          // {4,3,2,1}
lst.sort();             // {1,2,3,4}
lst.remove(2);          // removes all 2s
```

9.3 map (key-value, sorted)

Sorted by key · O(log n) insert/find/erase · unique keys · uses Red-Black Tree internally

```
// map_demo.cpp

#include <map>
map<string, int> marks;
marks["Alice"] = 95;
marks["Bob"]   = 87;
marks.insert({"Carol", 91});

cout << marks["Alice"];          // 95

for(auto& [key,val] : marks)
    cout << key << ":" << val << "\n";

if(marks.find("Dave") == marks.end())
    cout << "Dave not found";
```

9.4 set (unique sorted elements)

Stores unique elements · sorted automatically · O(log n) insert/find/erase

```
// set_demo.cpp

#include <set>
set<int> s = {5, 1, 3, 1, 5}; // {1,3,5}  duplicates removed
s.insert(4);          // {1,3,4,5}
s.erase(3);           // {1,4,5}

if(s.count(4)) cout << "4 exists";
if(s.find(9) == s.end()) cout << "9 not found";
```

Container	Header	Access	Insert	Erase	Use Case
vector	<vector>	O(1) random	O(1) back	O(n)	Array replacement, cache-friendly
list	<list>	O(n)	O(1) anywhere	O(1)	Frequent middle inserts/deletes
map	<map>	O(log n)	O(log n)	O(log n)	Key-value store, dictionary
set	<set>	O(log n)	O(log n)	O(log n)	Unique sorted elements
unordered_map	<unordered_map>	O(1) avg	O(1) avg	O(1) avg	Hash map, fast lookup

10. Iterators and Algorithms

10.1 Iterators

Iterators are generalised pointers that allow uniform traversal across all STL containers.

```
// iterators.cpp

vector<int> v = {10, 20, 30, 40, 50};

// begin() points to first element; end() points ONE PAST last
vector<int>::iterator it;
for(it = v.begin(); it != v.end(); ++it)
    cout << *it << " ";          // 10 20 30 40 50

// auto shorthand
for(auto it = v.begin(); it != v.end(); ++it)
    cout << *it;

// Reverse iterator
for(auto rit = v.rbegin(); rit != v.rend(); ++rit)
    cout << *rit;                    // 50 40 30 20 10
```

Iterator Type	Direction	Containers	Operators
input_iterator	Forward, read only	istream_iterator	++, *, ==, !=
output_iterator	Forward, write only	ostream_iterator	++, *
forward_iterator	Forward, read/write	forward_list	++, *, ==
bidirectional_iterator	Forward + backward	list, map, set	++, --, *
random_access_iterator	Any direction + jump	vector, deque	++, --, *, +n, -n, []

10.2 STL Algorithms (from <algorithm>)

```
// algorithms.cpp

#include <algorithm>
vector<int> v = {5, 2, 8, 1, 9, 3};

// Sorting
sort(v.begin(), v.end());                // {1,2,3,5,8,9} ascending
sort(v.begin(), v.end(), greater<int>());  // descending
```

```
// Searching
auto it = find(v.begin(), v.end(), 8);  // returns iterator

// Binary search (sorted container only)
bool found = binary_search(v.begin(), v.end(), 5);

// Count occurrences
int c = count(v.begin(), v.end(), 3);

// Reverse
reverse(v.begin(), v.end());

// Min / Max
auto mx = *max_element(v.begin(), v.end());
auto mn = *min_element(v.begin(), v.end());

// Apply function to all elements
for_each(v.begin(), v.end(), [](int x){ cout << x << " "; });
```

Important STL Algorithms

sort(begin, end) — O(n log n) introsort
find(begin, end, val) — O(n) linear search, returns iterator
binary_search(b, e, val) — O(log n), requires sorted range
count(begin, end, val) — count occurrences
reverse(begin, end) — in-place reversal
max_element / min_element — returns iterator to max/min
accumulate(b, e, init) — sum all elements (from <numeric>)
unique(begin, end) — remove consecutive duplicates
lower_bound / upper_bound — binary search boundaries

11. Exam Preparation

11.1 Important Long Answer Questions

C++ Basics & OOP Questions

1. Q1. Explain the structure of a C++ program. Describe the compilation pipeline from source code to executable.
2. Q2. What are constructors? Explain default, parameterised, and copy constructors with examples.
3. Q3. What is the difference between encapsulation and abstraction? Explain with a real-world C++ example.
4. Q4. Explain the three access specifiers in C++ with examples. What is the default access in a class?
5. Q5. Compare pointers and references in C++. Give examples of when each should be preferred.

Functions & Templates Questions

6. Q6. What is function overloading? Write a C++ program demonstrating overloaded area() functions.
7. Q7. Differentiate between inline functions and macros. When should inline be preferred?
8. Q8. Explain function templates with an example. How does the compiler perform template instantiation?
9. Q9. Write a class template for a generic Stack supporting push(), pop(), and isEmpty().
10. Q10. What is template specialisation? Write an example where a template is specialised for bool.

STL Questions

11. Q11. Compare vector, list, map, and set in terms of time complexity, memory layout, and use cases.
12. Q12. Write a C++ program using map to count word frequency in a given sentence.
13. Q13. What are iterators? Explain the five iterator categories with examples.
14. Q14. Write a program using STL algorithms: sort, find, binary_search, and count on a vector.
15. Q15. What is the difference between map and unordered_map? When would you prefer each?

11.2 MCQ Practice

Q1. Which keyword is used to allocate memory dynamically in C++?

a) malloc	**b) new ✓**
c) alloc	d) create

Q2. How many constructors can a class have?

a) Only one	b) Exactly two

c) Multiple (overloaded) ✓	d) None

Q3. Which access specifier is default for class members?

a) public	b) protected
c) private ✓	d) internal

Q4. What is the output of: int x=5; int& r=x; r=10; cout<<x;

a) 5	**b) 10 ✓**
c) Compile error	d) Undefined

Q5. Which container provides O(1) amortised push_back?

a) list	b) set
c) vector ✓	d) map

Q6. What does delete[] do?

a) Deletes one element	**b) Frees array allocated by new[] ✓**
c) Clears the array	d) Resets pointer to NULL

Q7. template <typename T> — what is T?

a) A fixed type	**b) A type parameter ✓**
c) A variable	d) A keyword

Q8. Which header must be included for STL sort()?

a) <sort>	b) <vector>
c) <algorithm> ✓	d) <utility>

11.3 Common Mistakes to Avoid

Memory Management Mistakes

✗ Using new without delete → memory leak
✗ Using delete instead of delete[] for arrays → undefined behaviour
✗ Accessing a pointer after delete → dangling pointer
✗ Forgetting nullptr check before dereferencing a pointer

Constructor / Class Mistakes

✗ Forgetting to initialise member variables in constructors
✗ Defining copy constructor but forgetting to deep-copy pointer members
✗ Assuming default constructor exists when you've defined a parameterised one
✗ Writing a destructor but forgetting to free dynamically allocated members

Template & STL Mistakes

✗ Defining a template function in a .cpp file instead of header → linker errors
✗ Using [] on map to check existence — creates a default-value entry if key absent
✗ Calling delete[] on a single object allocated with new (and vice versa)
✗ Using v.end() as a valid element — it points ONE PAST the last element
✗ Using sort() on list — list has its own .sort() member (STL sort needs random access)

11.4 Unit-I Quick Revision Cheat Sheet

Topic	Key Syntax / Keyword	Remember
Dynamic allocation	new / delete	Pair every new with delete
Default constructor	ClassName() {}	Created by compiler only if no ctor defined
Inline function	inline retType fn()	Hint only; best for 1-5 line functions
Function overloading	Same name, diff params	Return type alone is NOT enough
Function template	template <typename T>	Defined in headers; compiler instantiates
Class template	template <typename T> class	Explicit type needed: Stack<int>
vector	#include <vector>	push_back O(1), random access O(1)
list	#include <list>	O(1) insert/delete, no random access
map	#include <map>	Sorted, O(log n); use [] carefully
set	#include <set>	Unique sorted; O(log n) all ops
sort	#include <algorithm>	Needs random-access iterator; O(n log n)
Iterator range	begin() to end()	end() is one-past-last, never dereference